

TRƯỜNG ĐẠI HỌC XÂY DỰNG HÀ NỘI
KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO

Bài tập lớn môn: An toàn bảo mật thông tin

Đề tài: Tìm hiểu về http và https

Nhóm 3

Thành viên :

Trần Quang Tiến	0025968	68IT3
Đoàn Đức Huy		68IT3
Bùi Tiến Đạt		68IT3

Giảng viên hướng dẫn : Th.S Nguyễn Việt Nhật

Hà Nội, 03/2025

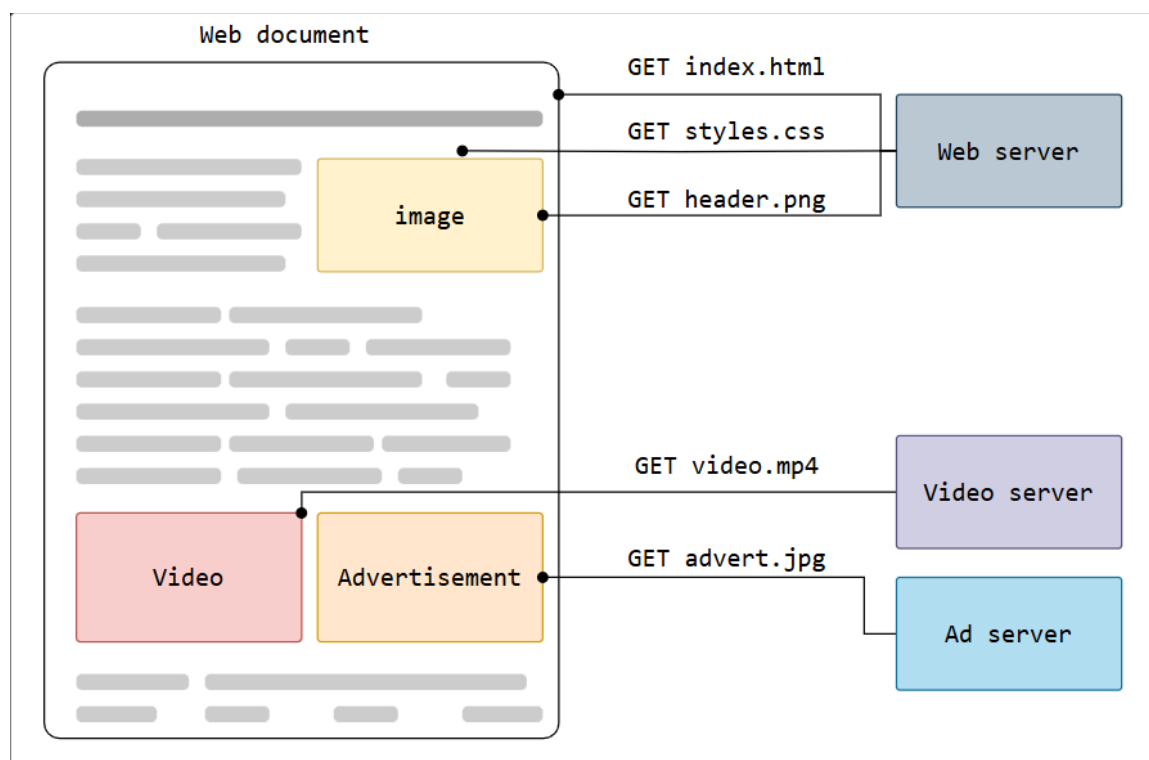
I. HTTP

1. HTTP là gì ?

- **HTTP (Hypertext Transfer Protocol)** là giao thức tầng ứng dụng dùng để truyền tải liệu siêu văn bản như HTML. Ban đầu được thiết kế cho việc giao tiếp giữa trình duyệt và máy chủ web, nhưng HTTP cũng được dùng cho các mục đích khác như giao tiếp giữa máy với máy hoặc truy cập API theo cách lập trình.
- HTTP hoạt động theo mô hình **client-server** cổ điển: client mở kết nối để gửi yêu cầu và chờ phản hồi từ server. Đây là một **giao thức không trạng thái (stateless)**, nghĩa là server không lưu thông tin phiên giữa các yêu cầu. Tuy nhiên, việc bổ sung **cookie** sau này cho phép duy trì trạng thái trong một số tương tác client-server.

2. TỔNG QUAN VỀ HTTP :

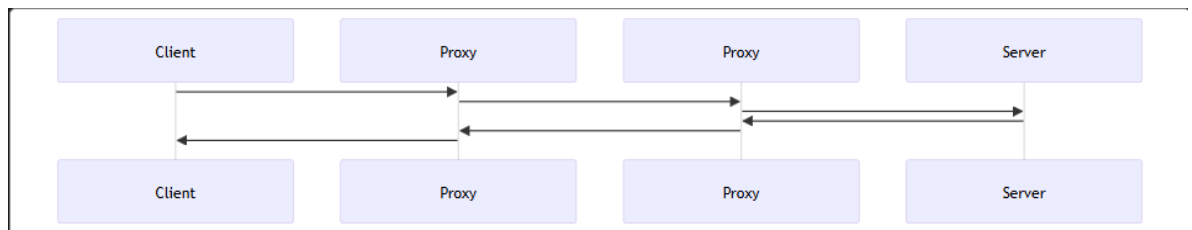
- HTTP là một giao thức dùng để truy xuất các tài nguyên như tài liệu HTML. Đây là nền tảng của mọi trao đổi dữ liệu trên Web và là một giao thức client-server, nghĩa là các yêu cầu được khởi tạo từ phía người nhận, thường là trình duyệt web. Một tài liệu hoàn chỉnh thường được cấu thành từ các tài nguyên như nội dung văn bản, hướng dẫn bố cục, hình ảnh, video, tập lệnh và nhiều thành phần khác. Các client và server giao tiếp với nhau bằng cách trao đổi các thông điệp riêng lẻ (khác với một luồng dữ liệu liên tục). Các thông điệp được gửi từ phía client được gọi là **yêu cầu (requests)**, còn các thông điệp được gửi từ phía server để phản hồi được gọi là **phản hồi (responses)**.



- Được thiết kế vào đầu những năm 1990, HTTP là một giao thức có thể mở rộng và đã phát triển theo thời gian. Đây là một giao thức thuộc tầng ứng dụng, được truyền qua TCP hoặc qua một kết nối TCP được mã hóa bằng TLS, mặc dù về lý thuyết, bất kỳ giao thức truyền tải đáng tin cậy nào cũng có thể được sử dụng. Nhờ khả năng mở rộng của nó, HTTP không chỉ được sử dụng để truy xuất các tài liệu siêu văn bản, mà còn để lấy hình ảnh, video hoặc gửi nội dung đến máy chủ, như kết quả từ các biểu mẫu HTML. HTTP cũng có thể được sử dụng để truy xuất các phần của tài liệu nhằm cập nhật các trang Web theo yêu cầu.

2.1 Các thành phần của hệ thống dựa trên HTTP

- HTTP là một giao thức theo mô hình client-server: các yêu cầu được gửi bởi một thực thể, gọi là **user-agent** (hoặc một proxy thay mặt nó). Phần lớn thời gian, user-agent là một trình duyệt web, nhưng nó cũng có thể là bất kỳ thứ gì, ví dụ như một robot thu thập dữ liệu trên Web để xây dựng và duy trì chỉ mục của công cụ tìm kiếm. Mỗi yêu cầu riêng lẻ được gửi đến một máy chủ, máy chủ này xử lý yêu cầu đó và cung cấp một câu trả lời gọi là **phản hồi** (response). Giữa client và server có nhiều thực thể trung gian, được gọi chung là **proxy**, thực hiện các chức năng khác nhau và có thể đóng vai trò là cổng trung gian hoặc bộ nhớ đệm (cache), chẳng hạn như vậy.



- Có các thiết bị như router, modem và nhiều phần tử mạng khác. Nhờ vào thiết kế phân tầng của Web, các phần tử này được ẩn ở tầng mạng (network) và tầng truyền tải (transport). HTTP nằm ở tầng trên cùng – tầng ứng dụng (application layer). Mặc dù các tầng bên dưới quan trọng đối với việc chẩn đoán sự cố mạng, chúng hầu như không liên quan đến mô tả của giao thức HTTP.

2.1.1 Client: User-Agent

- **User-agent** là bất kỳ công cụ nào hoạt động thay mặt cho người dùng. Vai trò này chủ yếu được thực hiện bởi **trình duyệt web**, nhưng cũng có thể là các chương trình được các kỹ sư hoặc nhà phát triển web sử dụng để kiểm tra lỗi ứng dụng.

- Để hiển thị một trang web, trình duyệt gửi một **yêu cầu ban đầu** để truy xuất tài liệu HTML đại diện cho trang đó. Sau đó nó phân tích tệp HTML này và thực hiện thêm các yêu cầu khác tương ứng với các tập lệnh thực thi, thông tin bố cục (CSS) để hiển thị, và các tài nguyên phụ chứa trong trang (thường là hình ảnh và video). Trình duyệt web sau đó kết hợp các tài nguyên này để hiển thị tài liệu hoàn chỉnh – **trang web**. Các tập lệnh được thực thi bởi trình duyệt có thể tiếp tục truy xuất thêm tài nguyên trong các giai đoạn tiếp theo, và trình duyệt sẽ cập nhật trang web tương ứng.

2.1.2 Web Server

- Ở phía bên kia của kênh truyền thông là **máy chủ (server)**, nơi phục vụ tài liệu theo yêu cầu từ client. Một máy chủ có thể **xuất hiện như một máy duy nhất**, nhưng thực tế có thể là **một tập hợp các máy chủ** chia sẻ tải (load balancing), hoặc kết hợp với các phần mềm khác như bộ nhớ đệm (cache), máy chủ cơ sở dữ liệu (database server), hoặc các máy chủ thương mại điện tử – tạo tài liệu hoàn toàn hoặc một phần theo yêu cầu.
- Một máy chủ **không nhất thiết là một máy duy nhất** – nhiều phần mềm máy chủ có thể được **lưu trữ trên cùng một máy vật lý**. Với HTTP/1.1 và header Host, chúng thậm chí có thể chia sẻ cùng một địa chỉ IP.

2.1.3 Proxy

- Giữa trình duyệt web và máy chủ, có **nhiều máy tính và thiết bị** truyền các thông điệp HTTP. Do kiến trúc phân tầng của Web stack, hầu hết các thiết bị này hoạt động ở tầng truyền tải, tầng mạng hoặc tầng vật lý – trở nên **trong suốt ở tầng HTTP**, nhưng có thể ảnh hưởng đáng kể đến hiệu suất.
- Những thiết bị hoạt động ở tầng ứng dụng thường được gọi là **proxy**. Proxy có thể là:
 - **Proxy trong suốt**: chuyển tiếp yêu cầu mà không thay đổi gì.
 - **Proxy không trong suốt**: thay đổi yêu cầu theo một cách nào đó trước khi gửi tiếp đến máy chủ.
- Proxy có thể thực hiện nhiều chức năng như:
 - **Bộ nhớ đệm (Caching)**: cache có thể công khai hoặc riêng tư (như cache của trình duyệt).
 - **Lọc (Filtering)**: ví dụ như quét virus hoặc kiểm soát nội dung cho phụ huynh.
 - **Cân bằng tải (Load balancing)**: cho phép nhiều máy chủ xử lý các yêu cầu khác nhau.
 - **Xác thực (Authentication)**: kiểm soát quyền truy cập vào các tài nguyên khác nhau.
 - **Ghi nhật ký (Logging)**: lưu lại thông tin lịch sử truy cập.

2.2 Những khía cạnh cơ bản của HTTP

2.2.1 HTTP đơn giản

- HTTP được thiết kế đơn giản và dễ đọc đối với con người, ngay cả khi những tính năng phức tạp hơn được giới thiệu trong HTTP/2 (chẳng hạn như việc đóng gói các thông điệp HTTP vào các khung – *frames*). Các thông điệp HTTP có thể được đọc và hiểu bởi con người, giúp việc kiểm thử dễ dàng hơn cho nhà phát triển, đồng thời giảm độ phức tạp cho người mới bắt đầu.

2.2.2 HTTP có khả năng mở rộng

- Khả năng mở rộng của HTTP được giới thiệu từ phiên bản HTTP/1.0, nhờ vào các header (tiêu đề). Điều này giúp giao thức trở nên dễ dàng để mở rộng và thử nghiệm. Các chức năng mới thậm chí có thể được đưa vào chỉ cần sự thỏa thuận giữa client và server về ý nghĩa (semantics) của một header mới.

2.2.3 HTTP và các kết nối

- Một kết nối được kiểm soát tại tầng truyền tải (*transport layer*), do đó nó nằm ngoài phạm vi xử lý trực tiếp của HTTP. HTTP không yêu cầu giao thức truyền tải bên dưới phải dựa trên kết nối (*connection-based*); nó chỉ yêu cầu sự đáng tin cậy, tức là không làm mất thông điệp (hoặc tối thiểu là phải báo lỗi khi điều đó xảy ra).
- Trong số hai giao thức truyền tải phổ biến nhất trên Internet:
 - **TCP** là đáng tin cậy.
 - **UDP** thì không.
- Do đó, HTTP dựa vào tiêu chuẩn **TCP**, vốn là giao thức dựa trên kết nối (*connection-based*).

2.3 Những gì có thể được kiểm soát bởi HTTP

- Tính chất mở rộng của HTTP theo thời gian đã cho phép kiểm soát nhiều hơn và bổ sung thêm chức năng cho Web. Một số chức năng như bộ nhớ đệm

(cache) và xác thực (authentication) đã được hỗ trợ từ sớm trong lịch sử phát triển HTTP. Trong khi đó, những khả năng như nói lỏng ràng buộc nguồn gốc (origin constraint) chỉ mới được thêm vào trong thập niên 2010. Dưới đây là danh sách các tính năng phổ biến có thể kiểm soát được thông qua HTTP:

➤ **Caching (Bộ nhớ đệm)**

HTTP cho phép điều khiển cách các tài liệu được lưu đệm:

- Máy chủ có thể ra lệnh cho các proxy và trình khách (client) về những gì nên được lưu đệm và trong bao lâu.
- Client cũng có thể yêu cầu các proxy trung gian bỏ qua tài liệu đã lưu đệm và yêu cầu phiên bản mới.

➤ **Relaxing the origin constraint (Nới lỏng ràng buộc nguồn gốc)**

Để ngăn chặn việc theo dõi hoặc vi phạm quyền riêng tư, trình duyệt web thực thi sự tách biệt nghiêm ngặt giữa các website:

- Chỉ các trang từ cùng một nguồn gốc (origin) mới có thể truy cập toàn bộ thông tin của một trang web.
- Tuy nhiên, nhờ vào các HTTP header, máy chủ có thể nới lỏng sự ngăn cách này, cho phép một tài liệu được ghép từ nhiều nguồn khác nhau.
- Trong một số trường hợp, việc nới lỏng này còn phục vụ mục đích bảo mật.

➤ **Authentication (Xác thực)**

Một số trang web được bảo vệ để chỉ người dùng cụ thể mới có quyền truy cập.

- HTTP hỗ trợ xác thực cơ bản (Basic Authentication) thông qua các header như `WWW-Authenticate`, hoặc
- Bằng cách thiết lập phiên làm việc (session) sử dụng HTTP cookies.

➤ **Proxy và tunneling (Đại diện và đường hầm)**

Client hoặc server thường nằm trong mạng nội bộ (intranet) và ẩn địa chỉ IP thật khỏi các máy khác.

- Trong trường hợp này, yêu cầu HTTP sẽ đi qua proxy để vượt qua rào cản mạng.
- Không phải tất cả proxy đều là proxy HTTP.
 - Ví dụ: giao thức SOCKS hoạt động ở cấp thấp hơn,
 - Hoặc các giao thức khác như FTP cũng có thể được xử lý bởi các proxy này.

➤ **Sessions (Phiên làm việc)**

Sử dụng HTTP cookies cho phép liên kết các yêu cầu HTTP với trạng thái của máy chủ, từ đó tạo ra các phiên làm việc (session) — mặc dù HTTP vốn là một giao thức không trạng thái.

- Điều này không chỉ hữu ích cho giỏ hàng trong thương mại điện tử,

- Mà còn áp dụng cho bất kỳ trang web nào cho phép người dùng tùy chỉnh nội dung đầu ra.

2.4 Kết luận

- HTTP là một giao thức có thể mở rộng và dễ sử dụng. Cấu trúc client-server, kết hợp với khả năng thêm các tiêu đề, cho phép HTTP tiến bộ cùng với những khả năng mở rộng của Web.
- Mặc dù HTTP/2 thêm một số phức tạp bằng cách nhúng các thông điệp HTTP vào các khung để cải thiện hiệu suất, nhưng cấu trúc cơ bản của các thông điệp vẫn giữ nguyên kể từ HTTP/1.0. Quy trình phiên làm việc vẫn cơ bản, cho phép nó được điều tra và gỡ lỗi với công cụ giám sát mạng HTTP.

3. SỰ TIẾN HÓA CỦA HTTP :

HTTP (HyperText Transfer Protocol) là giao thức cơ bản của World Wide Web. Được phát triển bởi Tim Berners-Lee và nhóm của ông trong khoảng thời gian từ 1989 đến 1991, HTTP đã trải qua nhiều thay đổi giúp duy trì sự đơn giản của nó đồng thời hình thành khả năng linh hoạt. Tiếp tục đọc để tìm hiểu cách HTTP tiến hóa từ một giao thức được thiết kế để trao đổi tệp tin trong môi trường phòng thí nghiệm bán tin cậy thành một mô hình hiện đại trên Internet, mang theo hình ảnh và video độ phân giải cao và 3D.

3.1 Sự phát minh của World Wide Web

- Vào năm 1989, khi đang làm việc tại CERN, Tim Berners-Lee đã viết một đề xuất để xây dựng một hệ thống siêu văn bản trên Internet. Ban đầu gọi là Mesh, sau đó được đổi tên thành World Wide Web trong quá trình triển khai vào năm 1990. Được xây dựng trên các giao thức TCP và IP hiện có, nó bao gồm 4 thành phần cơ bản:
 1. Một định dạng văn bản để đại diện cho các tài liệu siêu văn bản, Ngôn ngữ Đánh dấu Siêu văn bản (HTML).
 2. Một giao thức để trao đổi các tài liệu này, Giao thức Chuyển giao Siêu văn bản (HTTP).
 3. Một khách hàng để hiển thị (và chỉnh sửa) các tài liệu này, trình duyệt web đầu tiên gọi là WorldWideWeb.

4. Một máy chủ để cung cấp quyền truy cập vào tài liệu, một phiên bản đầu tiên của httpd.
- Bốn thành phần này đã hoàn thành vào cuối năm 1990, và những máy chủ đầu tiên đã hoạt động bên ngoài CERN vào đầu năm 1991. Vào ngày 6 tháng 8 năm 1991, Tim Berners-Lee đã đăng tải thông báo trên nhóm tin alt.hypertext công cộng. Đây hiện được coi là khởi đầu chính thức của World Wide Web như một dự án công cộng. Giao thức HTTP sử dụng trong những giai đoạn đầu rất đơn giản. Nó sau này được gọi là HTTP/0.9 và đôi khi được gọi là giao thức một dòng.

3.2 HTTP/0.9 – Giao thức một dòng

- Phiên bản ban đầu của HTTP không có số phiên bản; nó sau này được gọi là 0.9 để phân biệt với các phiên bản sau. HTTP/0.9 rất đơn giản: các yêu cầu chỉ bao gồm một dòng duy nhất và bắt đầu với phương thức duy nhất là GET, tiếp theo là đường dẫn đến tài nguyên. URL đầy đủ không được bao gồm vì giao thức, máy chủ và cổng không cần thiết khi đã kết nối với máy chủ.

```
HTTP
GET /my-page.html
```

- Phản hồi cũng rất đơn giản: nó chỉ bao gồm chính tệp tin đó.

```
HTML
<html>
  An text-only web page
</html>
```

- Khác với các sự tiến hóa sau này, HTTP/0.9 không có tiêu đề HTTP. Điều này có nghĩa là chỉ các tệp HTML mới có thể được truyền tải. Không có mã trạng thái hay mã lỗi. Nếu có vấn đề, một tệp HTML đặc biệt sẽ được tạo ra và bao gồm một mô tả về vấn đề để người dùng có thể hiểu được.

3.3 HTTP/1.0 – Xây dựng khả năng mở rộng

HTTP/0.9 rất hạn chế, nhưng các trình duyệt và máy chủ đã nhanh chóng làm cho nó trở nên linh hoạt hơn:

- Thông tin phiên bản được gửi trong mỗi yêu cầu (HTTP/1.0 được thêm vào dòng GET).
- Một dòng mã trạng thái cũng được gửi ở đầu phản hồi. Điều này cho phép trình duyệt tự nhận biết thành công hay thất bại của yêu cầu và điều chỉnh hành vi

của nó cho phù hợp. Ví dụ, cập nhật hoặc sử dụng bộ nhớ cache cục bộ theo một cách cụ thể.

- Khái niệm HTTP headers đã được giới thiệu cho cả yêu cầu và phản hồi. Metadata có thể được truyền tải và giao thức trở nên cực kỳ linh hoạt và có thể mở rộng.
- Các tài liệu ngoài các tệp HTML thuần túy có thể được truyền tải nhờ vào header Content-Type.

```
HTTP

GET /my-page.html HTTP/1.0
User-Agent: NCSA_Mosaic/2.0 (Windows 3.1)

HTTP/1.0 200 OK
Date: Tue, 15 Nov 1994 08:12:31 GMT
Server: CERN/3.0 libwww/2.17
Content-Type: text/html
<HTML>
A page with an image
  <IMG SRC="/my-image.gif">
</HTML>
```

- Giữa năm 1991 và 1995, các tính năng này được giới thiệu theo phương pháp thử và xem. Một máy chủ và trình duyệt sẽ thêm một tính năng và xem liệu nó có được chấp nhận hay không. Các vấn đề về tính tương thích giữa các hệ thống là rất phổ biến. Để giải quyết những vấn đề này, một tài liệu thông tin mô tả các thực hành chung đã được xuất bản vào tháng 11 năm 1996. Tài liệu này được gọi là RFC 1945 và đã định nghĩa HTTP/1.0.

3.4 HTTP/1.1 – Giao thức chuẩn hóa

- Trong khi đó, quá trình chuẩn hóa chính thức đang được tiến hành. Điều này diễn ra song song với các triển khai đa dạng của HTTP/1.0. Phiên bản HTTP chuẩn đầu tiên, HTTP/1.1, được công bố vào đầu năm 1997, chỉ vài tháng sau HTTP/1.0. HTTP/1.1 làm rõ các điểm mơ hồ và giới thiệu nhiều cải tiến:
 - Kết nối có thể được tái sử dụng, điều này tiết kiệm thời gian. Không còn cần phải mở kết nối nhiều lần để hiển thị các tài nguyên nhúng trong một tài liệu gốc duy nhất.
 - Pipelining được thêm vào. Điều này cho phép gửi yêu cầu thứ hai trước khi câu trả lời cho yêu cầu đầu tiên được truyền tải hoàn toàn. Điều này giảm độ trễ của giao tiếp.
 - Các cơ chế kiểm soát bộ nhớ cache bổ sung được giới thiệu.
 - Đàm phán nội dung, bao gồm ngôn ngữ, mã hóa và loại nội dung, được giới thiệu. Một khách hàng và một máy chủ có thể giờ đây đồng ý về loại nội dung cần trao đổi.
 - Dòng yêu cầu điển hình, tất cả qua một kết nối duy nhất, trông như thế này:

4. BỘ NHỚ ĐỆM HTTP

Bộ nhớ đệm HTTP giúp lưu trữ và tái sử dụng phản hồi để tăng tốc độ tải trang, giảm tải cho máy chủ và tối ưu hóa hiệu suất hệ thống.

4.1 Các loại bộ nhớ đệm

4.1.1 Bộ nhớ đệm riêng tư

- Bộ nhớ đệm riêng tư là bộ nhớ đệm liên kết với một khách hàng cụ thể — thường là bộ nhớ đệm của trình duyệt. Vì phản hồi đã lưu trữ không được chia sẻ với các khách hàng khác, bộ nhớ đệm riêng tư có thể lưu trữ các phản hồi cá nhân hóa cho người dùng đó.
- Ngược lại, nếu nội dung cá nhân hóa được lưu trữ trong một bộ nhớ đệm không phải bộ nhớ đệm riêng tư, thì các người dùng khác có thể truy cập những nội dung này, điều này có thể dẫn đến rò rỉ thông tin không mong muốn.

4.1.2 Bộ nhớ đệm chia sẻ

- Bộ nhớ đệm chia sẻ nằm giữa **khách hàng và máy chủ** và có thể lưu trữ các phản hồi có thể được chia sẻ giữa các người dùng. Bộ nhớ đệm chia sẻ có thể được phân loại thêm thành **bộ nhớ đệm proxy** và **bộ nhớ đệm quản lý**.

4.1.3 Bộ nhớ đệm proxy

- Ngoài chức năng kiểm soát quyền truy cập, một số proxy triển khai bộ nhớ đệm để giảm lưu lượng truy cập ra khỏi mạng. Điều này thường không được quản lý bởi nhà phát triển dịch vụ, vì vậy nó phải được kiểm soát thông qua các tiêu đề HTTP thích hợp và các phương thức khác. Tuy nhiên, trong quá khứ, các triển khai bộ nhớ đệm proxy lỗi thời — chẳng hạn như các triển khai không hiểu đúng chuẩn HTTP Caching — đã thường xuyên gây ra vấn đề cho các nhà phát triển.
- Các tiêu đề kitchen-sink như sau được sử dụng để vượt qua các triển khai bộ nhớ đệm proxy cũ và không được cập nhật, những cái không hiểu các chỉ thị hiện tại của chuẩn HTTP Caching như no-store.

4.1.4 Bộ nhớ đệm được quản lý

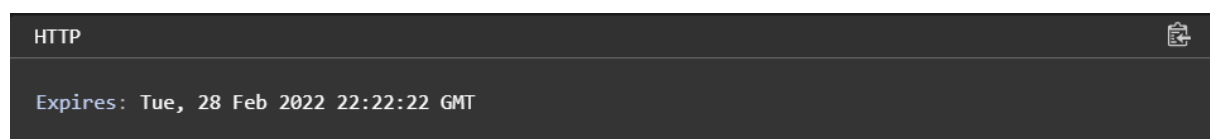
- Bộ nhớ đệm được quản lý là những bộ nhớ đệm được triển khai rõ ràng bởi các nhà phát triển dịch vụ để giảm tải cho máy chủ gốc và cung cấp nội dung hiệu quả. Các ví dụ bao gồm reverse proxy, CDN, và service workers kết hợp với Cache API.
- Đặc điểm của bộ nhớ đệm được quản lý thay đổi tùy thuộc vào sản phẩm được triển khai. Trong hầu hết các trường hợp, bạn có thể kiểm soát hành vi của bộ nhớ đệm thông qua Cache-Control header và các tệp cấu hình hoặc bảng điều khiển của riêng bạn.

4.2 Fresh and stale based on age

- Các phản hồi HTTP được lưu trữ có hai trạng thái: tươi (fresh) và cũ (stale). Trạng thái tươi thường chỉ ra rằng phản hồi vẫn còn hợp lệ và có thể tái sử dụng, trong khi trạng thái cũ có nghĩa là phản hồi đã hết hạn.
- Tiêu chí để xác định khi nào một phản hồi là tươi và khi nào là cũ là tuổi (age). Trong HTTP, tuổi là thời gian đã trôi qua kể từ khi phản hồi được tạo ra. Điều này tương tự với TTL trong các cơ chế bộ nhớ đệm khác.
- Đối với phản hồi ví dụ, ý nghĩa của `max-age` là như sau:
 - Nếu tuổi của phản hồi nhỏ hơn một tuần, phản hồi là tươi (fresh).
 - Nếu tuổi của phản hồi lớn hơn một tuần, phản hồi là cũ (stale).
 - Miễn là phản hồi đã lưu trữ vẫn còn tươi, nó sẽ được sử dụng để đáp ứng các yêu cầu từ khách hàng.
- Khi một phản hồi được lưu trữ trong bộ nhớ đệm chia sẻ, có thể thông báo cho khách hàng tuổi của phản hồi. Tiếp tục với ví dụ trên, nếu bộ nhớ đệm chia sẻ đã lưu trữ phản hồi trong một ngày, bộ nhớ đệm chia sẻ sẽ gửi phản hồi sau đây cho các yêu cầu tiếp theo từ khách hàng.

4.3 Expires or max-age

- Trong HTTP/1.0, tính tươi của phản hồi được chỉ định bởi header `Expires`.
- Header `Expires` chỉ định thời gian sống của bộ nhớ đệm bằng cách sử dụng một thời điểm cụ thể thay vì chỉ định thời gian trôi qua.



- Tuy nhiên, định dạng thời gian trong `Expires` khó phân tích, nhiều lỗi triển khai đã được phát hiện, và có thể gây ra vấn đề khi thay đổi đồng hồ hệ thống một cách cố ý; do đó, `max-age` — để chỉ định thời gian đã trôi qua — đã được áp dụng trong Cache-Control của HTTP/1.1.

- Nếu cả Expires và Cache-Control: max-age đều có mặt, thì max-age sẽ được ưu tiên. Vì vậy, hiện nay không cần phải cung cấp Expires nữa khi HTTP/1.1 đã được sử dụng rộng rãi.

4.4 Vary

- Tuy nhiên, nội dung của các phản hồi không phải lúc nào cũng giống nhau, ngay cả khi chúng có cùng một URL. Đặc biệt khi có sự đàm phán nội dung, phản hồi từ máy chủ có thể phụ thuộc vào giá trị của các tiêu đề yêu cầu như Accept, Accept-Language, và Accept-Encoding.
- Điều này khiến bộ nhớ đệm được xác định dựa trên sự kết hợp giữa URL của phản hồi và tiêu đề yêu cầu Accept-Language — thay vì chỉ dựa trên URL của phản hồi.
- Ngoài ra, nếu bạn đang cung cấp tối ưu hóa nội dung (ví dụ, cho thiết kế đáp ứng) dựa trên tác nhân người dùng (user agent), bạn có thể sẽ bị cám dỗ để thêm User-Agent vào giá trị của tiêu đề Vary. Tuy nhiên, tiêu đề yêu cầu User-Agent thường có rất nhiều biến thể, điều này làm giảm đáng kể khả năng bộ nhớ đệm sẽ được tái sử dụng. Vì vậy, nếu có thể, thay vì dựa trên User-Agent, hãy cân nhắc một cách khác để thay đổi hành vi dựa trên phát hiện tính năng.

4.5 Validation

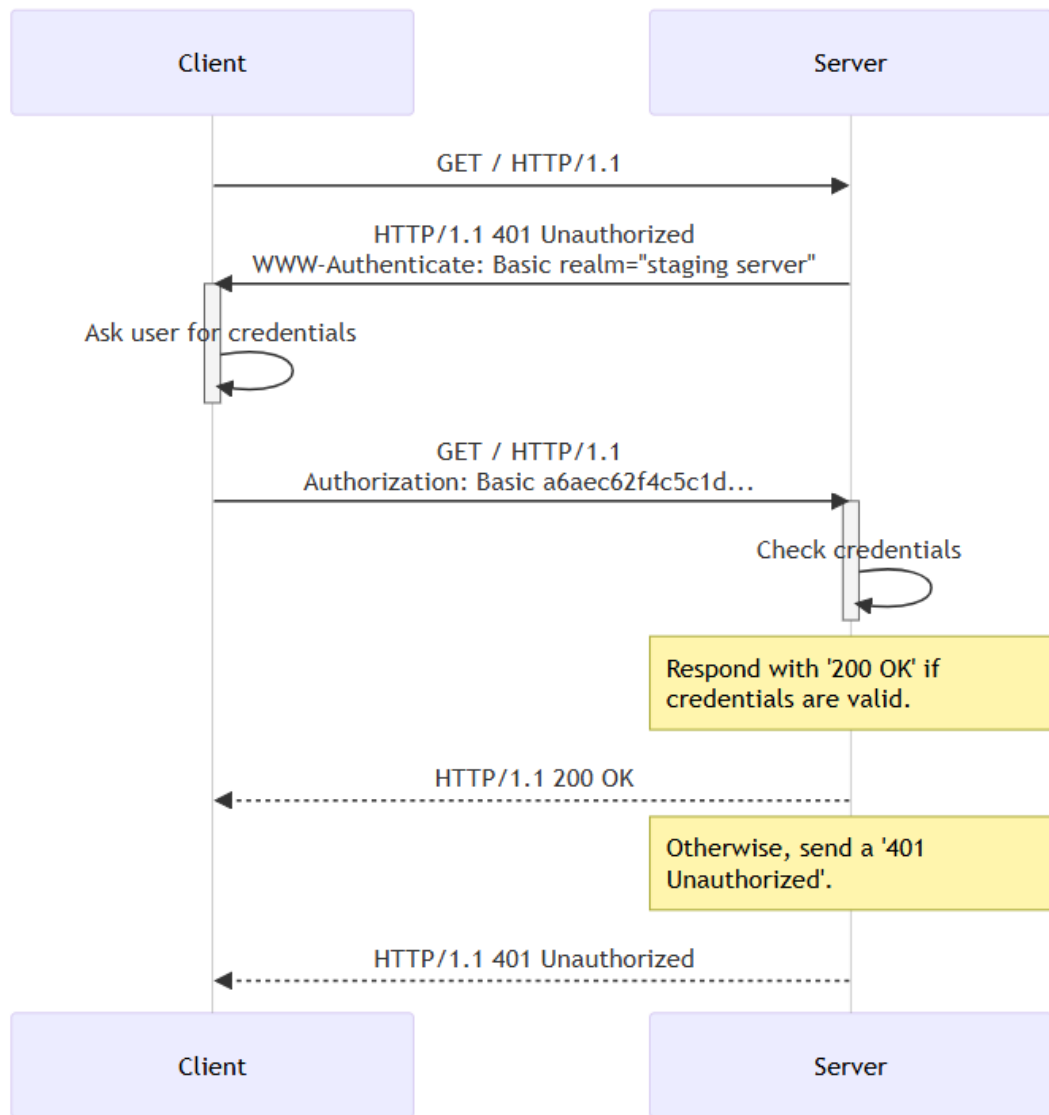
- Các phản hồi cũ (stale) không bị loại bỏ ngay lập tức. HTTP có cơ chế để chuyển một phản hồi cũ thành phản hồi mới bằng cách yêu cầu máy chủ gốc xác nhận lại. Cơ chế này được gọi là xác thực (validation), hoặc đôi khi là tái xác thực (revalidation).
- Xác thực được thực hiện bằng cách sử dụng một yêu cầu có điều kiện, bao gồm tiêu đề yêu cầu If-Modified-Since hoặc If-None-Match.

5. XÁC THỰC HTTP

- HTTP cung cấp một khung làm việc chung cho việc kiểm soát truy cập và xác thực. Trang này là phần giới thiệu về khung xác thực của HTTP, và trình bày cách giới hạn quyền truy cập vào máy chủ của bạn bằng cách sử dụng phương thức "Cơ bản" (Basic) của HTTP.

5.1 Khung xác thực chung của HTTP

- RFC 7235 định nghĩa khung xác thực HTTP, có thể được sử dụng bởi máy chủ để đưa ra yêu cầu xác thực đối với một yêu cầu từ phía máy khách, và cũng được sử dụng bởi máy khách để cung cấp thông tin xác thực.
- Quy trình thử thách và phản hồi hoạt động như sau:
 - Máy chủ phản hồi lại máy khách bằng mã trạng thái 401 (Không được phép) và cung cấp thông tin về cách xác thực thông qua tiêu đề phản hồi WWW-Authenticate, trong đó chứa ít nhất một yêu cầu xác thực (challenge).
 - Máy khách muốn tự xác thực với máy chủ có thể thực hiện điều này bằng cách thêm tiêu đề yêu cầu Authorization cùng với thông tin xác thực (credentials).
 - Thông thường, máy khách sẽ hiển thị một hộp thoại yêu cầu nhập mật khẩu cho người dùng, sau đó gửi lại yêu cầu bao gồm tiêu đề Authorization phù hợp.



5.2 Các phương thức xác thực

- Khung xác thực HTTP tổng quát là nền tảng cho nhiều phương thức xác thực khác nhau.
- IANA duy trì một danh sách các phương thức xác thực, nhưng cũng có những phương thức khác được cung cấp bởi các dịch vụ lưu trữ, chẳng hạn như Amazon AWS.
- Một số phương thức xác thực phổ biến bao gồm:
 - **Basic** : Xem RFC 7617, sử dụng thông tin xác thực được mã hóa base64. Thông tin chi tiết hơn sẽ có bên dưới.
 - **Bearer** : Xem RFC 6750, sử dụng token bearer để truy cập các tài nguyên được bảo vệ bởi OAuth 2.0.
 - **Digest** : Xem RFC 7616. Firefox từ phiên bản 93 trở đi hỗ trợ thuật toán SHA-256. Các phiên bản trước chỉ hỗ trợ mã băm MD5 (không được khuyến khích).
 - **HOBA** : Xem RFC 7486, Mục 3, Xác thực HTTP dựa trên nguồn gốc, sử dụng chữ ký số.
 - **Mutual** : Xem RFC 8120.
 - **Negotiate / NTLM** : Xem RFC 4599.
 - **VAPID** : Xem RFC 8292.
 - **SCRAM** : Xem RFC 7804.
 - **AWS4-HMAC-SHA256** : Xem tài liệu của AWS. Phương thức này được sử dụng cho xác thực máy chủ của AWS3.
- Các phương thức có thể khác nhau về mức độ bảo mật và khả năng tương thích với phần mềm máy khách hoặc máy chủ.
- Phương thức xác thực "Basic" cung cấp mức độ bảo mật rất kém, nhưng lại được hỗ trợ rộng rãi và dễ dàng thiết lập. Phương thức này sẽ được giới thiệu chi tiết hơn ở phần dưới.

5.3 Phương thức xác thực Basic

- Phương thức xác thực HTTP "Basic" được định nghĩa trong RFC 7617, trong đó thông tin xác thực được truyền dưới dạng cặp tên người dùng/mật khẩu, được mã hóa bằng base64.

5.3.1 Tính bảo mật của xác thực Basic

- Vì tên người dùng và mật khẩu được truyền qua mạng dưới dạng văn bản rõ (mặc dù được mã hóa bằng base64, nhưng base64 là một dạng mã hóa có thể đảo ngược), nên phương thức xác thực Basic không an toàn. HTTPS/TLS nên được sử dụng cùng với xác thực Basic. Nếu không có các biện pháp bảo mật bổ sung này, thì không nên sử dụng xác thực Basic để bảo vệ thông tin nhạy cảm hoặc có giá trị.
- Hạn chế truy cập bằng Apache và xác thực Basic. Để bảo vệ thư mục bằng mật khẩu trên máy chủ Apache, bạn sẽ cần một tệp .htaccess và một tệp .htpasswd.
- Tệp .htaccess thường có nội dung như sau:

```

APACHECONF
AuthType Basic
AuthName "Access to the staging site"
AuthUserFile /path/to/.htpasswd
Require valid-user

```

- Tệp .htaccess tham chiếu đến một tệp .htpasswd, trong đó mỗi dòng bao gồm một tên người dùng và một mật khẩu được phân tách bằng dấu hai chấm (:). Bạn không thể nhìn thấy mật khẩu thực vì chúng đã được băm (trong trường hợp này là sử dụng thuật toán băm dựa trên MD5). Lưu ý rằng bạn có thể đặt tên tệp .htpasswd theo ý muốn, nhưng cần nhớ rằng tệp này không nên để người khác truy cập được. (Apache thường được cấu hình để ngăn chặn truy cập vào các tệp bắt đầu bằng .ht*).

```

APACHECONF
aladdin:$apr1$ZjTqBB3f$IF9gdYAGlMrs2fuINjHsz.
user2:$apr1$004r.y2H$/vEkesPhVInBBYJukXitA/

```

5.3.2 Hạn chế truy cập bằng Nginx và xác thực Basic

- Đối với Nginx, bạn cần chỉ định một vị trí (location) mà bạn muốn bảo vệ, cùng với chỉ thị auth_basic dùng để đặt tên cho khu vực được bảo vệ bằng mật khẩu. Chỉ thị auth_basic_user_file sau đó sẽ trỏ đến tệp .htpasswd chứa thông tin người dùng đã được mã hóa, tương tự như trong ví dụ với Apache ở trên.

```

APACHECONF
location /status {
    auth_basic "Access to the staging site";
    auth_basic_user_file /etc/apache2/.htpasswd;
}

```

5.3.3 Truy cập bằng thông tin xác thực trong URL

- Nhiều ứng dụng khách (client) cũng cho phép bạn tránh hộp thoại đăng nhập bằng cách sử dụng một URL đã mã hóa, chứa tên người dùng và mật khẩu, như sau:

`https://username:password@www.example.com/`

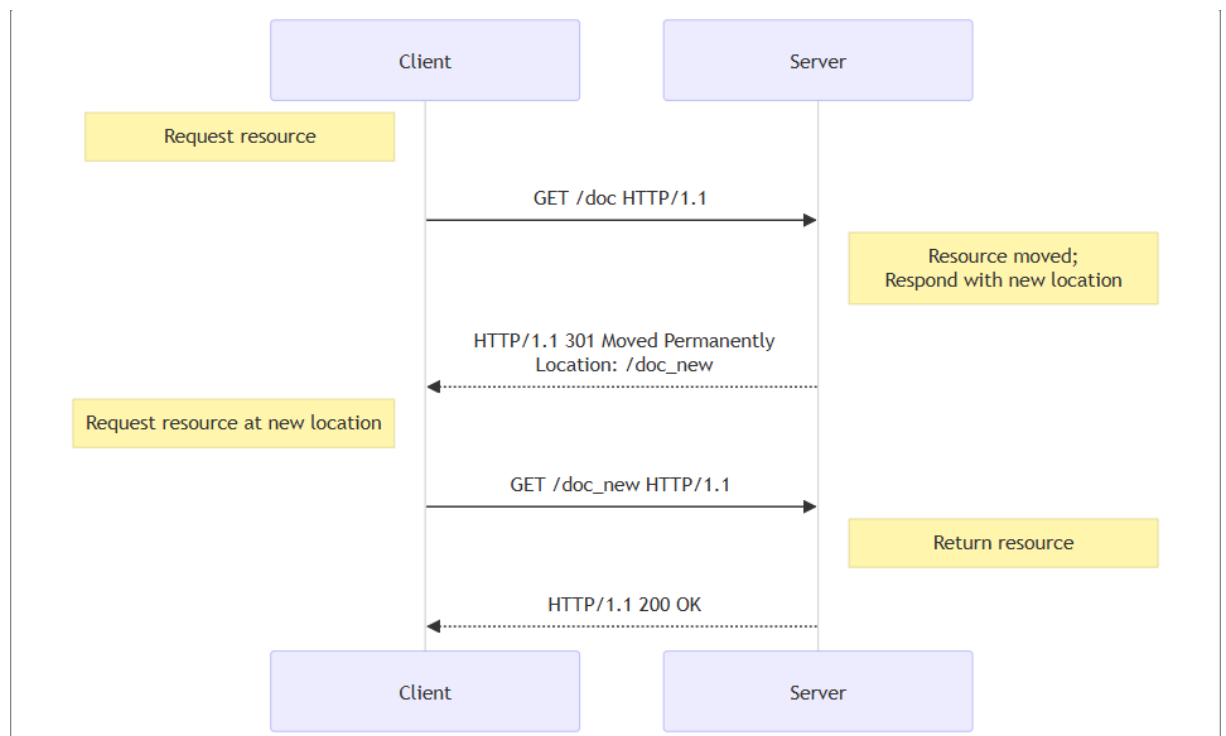
- Việc sử dụng các URL kiểu này hiện đã không còn được khuyến khích. Trên trình duyệt Chrome, phần `username:password@` trong URL sẽ bị loại bỏ khỏi các URL phụ (subresource URLs) vì lý do bảo mật. Trong Firefox, trình duyệt sẽ kiểm tra xem trang web đó có thực sự yêu cầu xác thực hay không. Nếu không, Firefox sẽ cảnh báo người dùng bằng một hộp thoại:
"Bạn sắp đăng nhập vào trang www.example.com với tên người dùng là `username`, nhưng trang web này không yêu cầu xác thực. Đây có thể là một hành vi cố tình đánh lừa bạn."
Trong trường hợp trang web có yêu cầu xác thực, Firefox vẫn sẽ yêu cầu người dùng xác nhận:
"Bạn sắp đăng nhập vào trang www.example.com với tên người dùng là `username`."
trước khi gửi thông tin xác thực đến trang web.
- Lưu ý rằng trong cả hai trường hợp, Firefox sẽ gửi yêu cầu không kèm theo thông tin xác thực trước, để xác định liệu trang web có yêu cầu xác thực hay không, rồi mới hiển thị hộp thoại thông báo.

6. CHUYỂN HƯỚNG TRONG HTTP

- Chuyển hướng URL, hay còn gọi là chuyển tiếp URL, là một kỹ thuật để cung cấp nhiều địa chỉ URL cho một trang, một biểu mẫu, toàn bộ trang web hoặc một ứng dụng web. HTTP có một loại phản hồi đặc biệt, được gọi là chuyển hướng HTTP, cho thao tác này.
- Chuyển hướng thực hiện nhiều mục tiêu:
 - Chuyển hướng tạm thời trong thời gian bảo trì hoặc ngừng hoạt động của trang web
 - Chuyển hướng vĩnh viễn để duy trì các liên kết/đánh dấu trang đã có sau khi thay đổi URL của trang web, chuyển tiếp các trang trong quá trình tải lên tệp, v.v.

6.1 Nguyên lý

- Trong HTTP, chuyển hướng được kích hoạt khi máy chủ gửi một phản hồi chuyển hướng đặc biệt cho một yêu cầu. Các phản hồi chuyển hướng có mã trạng thái bắt đầu bằng số 3, và một tiêu đề **Location** chứa URL cần chuyển hướng đến.
- Khi trình duyệt nhận được một phản hồi chuyển hướng, chúng sẽ ngay lập tức tải URL mới được cung cấp trong tiêu đề **Location**. Bên cạnh việc có một chút giảm hiệu suất do phải thực hiện một vòng đi-về bổ sung, người dùng hiếm khi nhận thấy sự chuyển hướng này.



6.2 Cách thay thế để chỉ định chuyển hướng

- Chuyển hướng HTTP không phải là cách duy nhất để định nghĩa chuyển hướng. Có hai cách khác:
 1. Chuyển hướng HTML bằng thẻ <meta>
 2. Chuyển hướng JavaScript qua DOM

6.2.1 Chuyển hướng HTML

- Chuyển hướng HTTP là cách tốt nhất để tạo chuyển hướng, nhưng đôi khi bạn không có quyền kiểm soát máy chủ. Trong trường hợp đó, bạn có thể thử sử dụng

dùng thẻ <meta> với thuộc tính **http-equiv** được thiết lập thành **Refresh** trong phần <head> của trang. Khi trình duyệt hiển thị trang, nó sẽ chuyển đến URL được chỉ định.

```
HTML
<head>
  <meta http-equiv="Refresh" content="0; URL=https://example.com/" />
</head>
```

- Thuộc tính **content** nên bắt đầu bằng một số cho biết số giây mà trình duyệt nên đợi trước khi chuyển hướng đến URL đã cho. Luôn đặt giá trị là 0 để tuân thủ quy chuẩn về khả năng truy cập.
- Rõ ràng, phương pháp này chỉ hoạt động với HTML và không thể được sử dụng cho hình ảnh hoặc các loại nội dung khác.

6.2.2 Chuyển hướng JavaScript

- Chuyển hướng trong JavaScript được thực hiện bằng cách gán một chuỗi URL vào thuộc tính **window.location**, để tải trang mới:

```
JS
window.location = "https://example.com/";
```

- Giống như chuyển hướng HTML, phương pháp này không thể hoạt động trên tất cả các tài nguyên, và rõ ràng, nó chỉ hoạt động trên các khách hàng thực thi JavaScript. Mặt khác, phương pháp này có nhiều khả năng hơn: ví dụ, bạn có thể kích hoạt chuyển hướng chỉ khi một số điều kiện nhất định được đáp ứng.

6.3 Các trường hợp sử dụng

- Có rất nhiều trường hợp sử dụng cho chuyển hướng, nhưng vì hiệu suất bị ảnh hưởng mỗi khi có một chuyển hướng, việc sử dụng chúng nên được giữ ở mức tối thiểu.

Alias tên miền

Lý tưởng nhất là có một vị trí, và do đó một URL, cho mỗi tài nguyên. Tuy nhiên, có một số lý do cho các tên thay thế cho một tài nguyên:

- **Mở rộng phạm vi tiếp cận của trang web**
Một trường hợp phổ biến là khi một trang web nằm tại www.example.com, nhưng việc truy cập từ example.com cũng nên hoạt động. Do đó, các chuyển hướng từ example.com tới www.example.com được thiết lập. Bạn cũng có thể

chuyển hướng từ các từ đồng nghĩa phổ biến hoặc các lỗi chính tả thường gặp của tên miền của mình.

- **Chuyển sang một tên miền mới**

Ví dụ, công ty của bạn đã đổi tên, nhưng bạn muốn các liên kết hoặc đánh dấu trang hiện có vẫn có thể tìm thấy bạn dưới tên mới.

- **Buộc sử dụng HTTPS**

Các yêu cầu đối với phiên bản http:// của trang web của bạn sẽ chuyển hướng tới phiên bản https://.

- **Giữ các liên kết hoạt động**

Khi bạn tái cấu trúc trang web, URL sẽ thay đổi. Ngay cả khi bạn cập nhật các liên kết trên trang web của mình để khớp với các URL mới, bạn không thể kiểm soát các URL được sử dụng bởi các tài nguyên bên ngoài.

Bạn không muốn làm hỏng những liên kết này, vì chúng mang lại người dùng có giá trị và giúp cải thiện SEO của bạn, vì vậy bạn thiết lập chuyển hướng từ các URL cũ sang các URL mới.

- **Các phản hồi tạm thời đối với các yêu cầu không an toàn**

Các yêu cầu không an toàn sẽ thay đổi trạng thái của máy chủ và người dùng không nên gửi lại chúng một cách vô tình.

- Thông thường, bạn không muốn người dùng gửi lại các yêu cầu PUT, POST hoặc DELETE. Nếu bạn phục vụ phản hồi là kết quả của yêu cầu này, một lần nhấn nút tải lại sẽ gửi lại yêu cầu (có thể sau một thông báo xác nhận).

- Trong trường hợp này, máy chủ có thể gửi lại phản hồi 303 (See Other) đến một URL chứa thông tin đúng. Nếu nút tải lại được nhấn, chỉ trang đó sẽ được hiển thị lại, mà không gửi lại các yêu cầu không an toàn.

- **Phản hồi tạm thời đối với các yêu cầu dài**

Một số yêu cầu có thể cần thêm thời gian trên máy chủ, như các yêu cầu DELETE được lên lịch để xử lý sau. Trong trường hợp này, phản hồi sẽ là một chuyển hướng 303 (See Other) liên kết đến một trang thông báo rằng hành động đã được lên lịch, và cuối cùng thông báo về tiến trình của nó hoặc cho phép hủy bỏ.